

Laboratoire 5

Profiling

Départements : TIC
Unité d'enseignement HPC

Auteur : Urs Behrmann
Professeur : Alberto Dassatti
Assistant : Bruno Da Rocha Carvalho
Classe : HPC
Salle de labo : A09
Date : 4 mai 2026

Table des matières

1	Analyse du programme fourni	3
1.1	Scénario et outils	3
1.2	Résultat global	3
1.3	Résultat <code>callgrind</code>	3
1.4	Lien avec <code>perf</code>	4
1.5	Piste d'amélioration	4
2	Analyse du programme du laboratoire 01	5
2.1	Scénario	5
2.2	Ce que j'ai vu avec <code>callgrind</code>	5
2.3	Problème dans mon code	5
2.4	Changement que j'ai fait	5
2.5	Avant / après	5

1 Analyse du programme fourni

1.1 Scénario et outils

J'ai compilé le projet dans `lab05/build`. J'ai généré **200 000** lignes (fichier `measurements.txt`) avec `./create_sample 200000`, puis j'ai profilé l'exécutable `./main measurements.txt`.

Le sujet demande d'utiliser les outils vus en cours. J'ai fait au minimum :

- **perf** : `perf stat` pour une vue globale (cycles, instructions, etc.) et `perf record -g + perf report` pour voir quelles fonctions coûtent quand ça marche sur la machine (chez moi : `sudo perf ...`).
- **Valgrind** / **callgrind** : pour compter les instructions par fonction de façon reproductible, même si le programme tourne beaucoup plus lentement qu'en natif.

1.2 Résultat global

Exemple de commandes (dans `build/`, après compilation) :

```
./create_sample 200000  
/usr/bin/time -p ./main measurements.txt  
sudo perf stat ./main measurements.txt  
sudo perf record -g ./main measurements.txt  
sudo perf report
```

Avec `time`, sur mon PC, `./main measurements.txt` prend environ **0,11 s** de temps réel pour 200 000 lignes (presque tout en temps utilisateur). Ça donne une idée de l'ordre de grandeur avant de lancer Valgrind.

1.3 Résultat callgrind

Commandes :

```
valgrind --tool=callgrind ./main measurements.txt  
callgrind_annotate callgrind.out.<pid>
```

D'après `callgrind_annotate`, l'essentiel des instructions (**Ir**) du programme passe dans trois zones :

Zone (symbole / fichier)	Part des Ir
<code>__strcmp_avx2</code> (libc)	≈ 55 %
<code>main</code> (lab05/src/main.c)	≈ 28 %
<code>strtod</code> et fonctions liées (libc)	≈ 7 %

Les appels à `strcmp` viennent surtout de la recherche de ville : dans le code, `getcitty` parcourt le tableau `results` et compare les noms avec `strcmp` jusqu'à trouver la bonne entrée. Comme on refait ça pour **chaque ligne** du fichier, le coût explique la grosse part de `strcmp`. `strtod` correspond au parsing de la température après le `;`. Le tri `qsort` à la fin est beaucoup plus petit ici par rapport au reste (peu de villes différentes par rapport au nombre de lignes).

1.4 Lien avec perf

Avec `sudo perf record -g ./main measurements.txt` puis `perf report`, on devrait retomber sur la même idée : beaucoup de temps dans les comparaisons de chaînes / `getcity` et dans le parsing, plutôt que dans le tri final.

1.5 Piste d'amélioration

Une table de hachage (clé = nom de ville) ou une structure qui évite de scanner tout le tableau à chaque ligne réduirait directement le coût observé sur `strcmp`, au lieu d'optimiser une partie du code qui ne domine pas.

2 Analyse du programme du laboratoire 01

2.1 Scénario

J'ai utilisé le fichier `80bpm0.csv` du dossier du labo (7500 échantillons). Commande typique :

```
./ecg_dealination ../80bpm0.csv sortie.json
```

2.2 Ce que j'ai vu avec callgrind

Avec Valgrind, une grosse partie des instructions vient de `ecg_analyze`, mais aussi du CSV (`strtod`, etc.). Ça correspond à l'idée que le traitement du signal coûte, et que le format texte coûte aussi.

2.3 Problème dans mon code

Dans `ecg_analyze`, pour calculer le seuil flottant, j'avais une boucle sur chaque échantillon et **à l'intérieur** deux boucles sur toute la fenêtre autour de l'échantillon pour calculer moyenne et écart-type. Quand la fenêtre est large (environ 600 ms de signal de chaque côté, donc beaucoup de points), ça fait énormément de travail répété : pour chaque index, je recalculais tout depuis zéro au lieu de réutiliser ce qui ne change qu'un peu.

2.4 Changement que j'ai fait

J'ai appris qu'on peut préparer deux tableaux "cumulés" : un pour la somme des valeurs, un pour la somme des carrés. Comme ça, pour une fenêtre donnée, la somme sur la fenêtre se récupère en soustrayant deux cases du tableau cumulé (comme quand on fait une aire sous courbe par morceaux). L'écart-type se déduit de la somme et de la somme des carrés. Du coup le calcul du seuil devient beaucoup moins naïf.

2.5 Avant / après

Sur le même CSV, le programme affiche le temps passé dans `ecg_analyze`. Avant, j'étais plutôt autour de **13 ms**. Après la modification, c'est autour de **0,3 ms**. Le nombre de pics R détectés (96) n'a pas changé, donc je considère que le résultat reste correct sur ce test.